The Journal of Open Source Software

CryoStack-v1.0.0: A modular cyberinfrastructure stack for cryosphere data, models, data assimilation, and heterogeneous computing

Brian Kyanjo [1¶], **Alexander A. Robel** [1], **Angelo S. Tarzona** [1], **Eliza J. Dawson** [1], **and Winnie Chu** [1]

1 School of Earth and Atmospheric Sciences, Georgia Institute of Technology, Atlanta, GA, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 21 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

CryoStack is an open-source cyberinfrastructure stack for assembling reproducible cryosphere workflows from applications, workspaces, data catalogs, simulation configurations, and heterogeneous computing resources. It grew from ice-sheet data-assimilation deployment tooling into a shared platform for modeling, ensemble assimilation, and radar observations. Its applications are CryoLauncher, ICESEE (Kyanjo, Mayo, et al., 2026), LIVIST (Living Ice Sheet Temperature), and Frozen Legacies. Each retains its domain-specific interface while reusing shared services where applicable. Supported workflows use local or institutional resources, or Cloud execution currently backed by Amazon Web Services (AWS). CryoStack is available at https://cryostack.eas.gatech.edu/; source code is at https://github.com/ICESEE-project/CryoStack.

Statement of need

Cryosphere research combines large datasets, interactive interpretation, numerical models, inverse methods, and ensemble simulations. Interactive discovery and quality control, workstation development, and production simulations require different environments, including Message Passing Interface (MPI)-enabled libraries, batch schedulers, and programmatic cloud access. Historical radar collections additionally require specialized ingestion, geolocation, quality control, and provenance.

Researchers often connect separate repositories manually: installing compiled dependencies, translating formats, writing scheduler scripts, transferring inputs, tracking jobs, retrieving outputs, and recording parameters and environments. This burden recurs across models, datasets, institutions, and computing systems, particularly affecting students and groups without dedicated research-software or high-performance computing (HPC) support. CryoStack serves researchers moving between exploratory and production workflows, collaborators using different environments, radar scientists preserving difficult legacy observations, and instructors introducing ice-sheet modeling and analysis. Its goal is explicit, repeatable, and inspectable transitions among applications, data, and resources.

State of the field

ISSM (Ice-sheet and Sea-level System Model) (Larour et al., 2012) and Icepack (Shapero et al., 2021) model ice-sheet flow and evolution at complementary complexity levels in different software environments. ICESEE (Kyanjo, Mayo, et al., 2026) provides ensemble state

DRAFT

and parameter estimation. Observational applications instead emphasize dataset discovery, preservation, geolocation, interpretation, and visualization, initially for ice-penetrating radar and borehole data.

CryoStack supplies the integration layer connecting these specialized capabilities: user access, persistent workspaces, configuration, data transfer, experiment tracking, remote computing, and deployment. It reuses scientific algorithms and interfaces rather than replacing them, allowing applications and computing environments to evolve independently.

JupyterHub serves multi-user notebooks (Project Jupyter Contributors, n.d.); Open OnDemand provides browser access to HPC (Hudak et al., 2018). CryoStack specializes integration around model-aware configuration, workflow capabilities, licensing, qualified execution paths, and cryosphere data ingestion.

Software design

CryoStack separates access and operations, scientific applications, execution backends, and reproducible environments (Figure 1). Experiments, datasets, and applications are related but independent objects. Identity and persistence reside outside applications; containment checks enforce per-user workspace roots. Application registrations declare deployable services, while a capability registry describes configuration subsets, result packages, licensing, and execution support. Scientific configurations move across supported environments; resource settings and readiness checks remain backend-specific. Connectivity and onboarding are shared where applicable, without imposing universal execution pipelines.

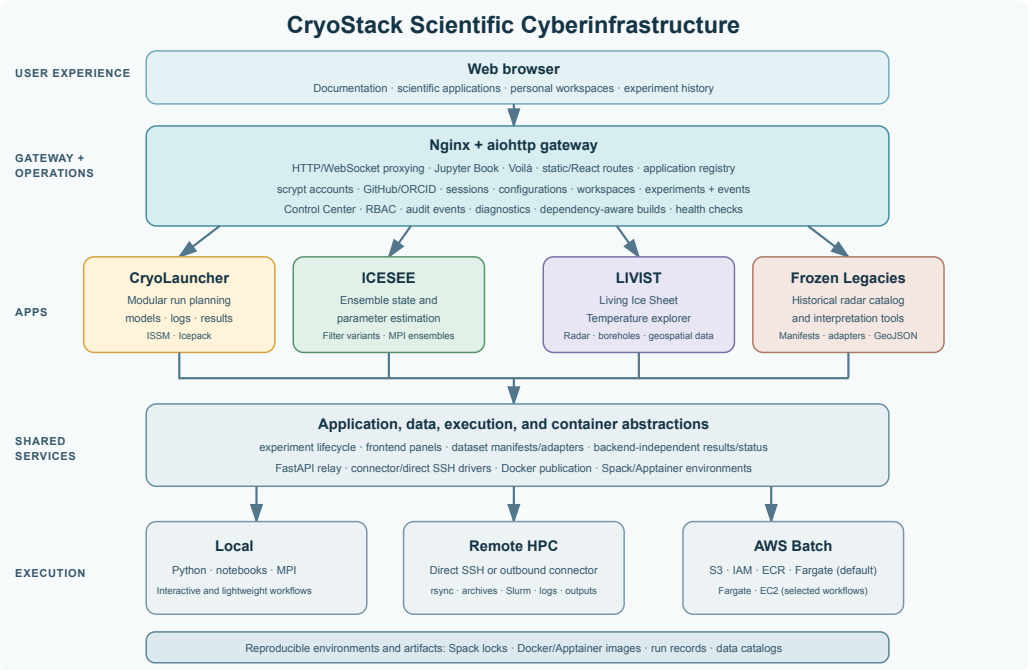


Figure 1: CryoStack's layered architecture. Shared services support application-specific workflows; Local, Remote-HPC, and AWS Batch execution do not apply to every application.

SQLite (Gaffney et al., 2022) stores users, sessions, saved configurations, per-application workspace state, and experiments. Each experiment records an immutable configuration snapshot with its application, backend, job identifiers, output paths, and a status-event timeline. Canonical examples remain read-only and are copied into user-owned working copies before editing or execution. Users can resume workspaces, reuse named configurations, and

relate scheduler state to its producing configuration. These records support, but do not complete, archival provenance of inputs, environments, transformations, and outputs.

Scientific applications

CryoLauncher configures and runs ISSM and Icepack. Basic mode exposes a curated, solver-aware parameter subset through dropdown menus and parameter inputs; Advanced provides access to the full working configuration scripts. Both use the same configuration. Remote and Cloud execution use institutional HPC and AWS. Supported ISSM workflows require MATLAB licensing; Icepack uses Firedrake-based notebooks or scripts without proprietary dependencies.

ICESEE provides ensemble-based state and parameter estimation (Kyanjo, Mayo, et al., 2026), coupling with ISSM, Icepack, and test implementations of Lorenz-96 and a one-dimensional glacier flowline model (Kyanjo, Robel, & Mayo, 2026). Dependencies vary by model. ICESEE supports Local, Remote, and Cloud execution through its own submission and result-handling implementation. It reuses shared connectivity and cloud onboarding where applicable; assimilation diagnostics retain their own structure outside CryoStack's common result conventions.

Frozen Legacies supports preservation and reuse of historical Antarctic ice-penetrating radar observations (Schroeder et al., 2019). Dataset manifests describe collections and metadata; ingestion adapters generate catalogs of geolocated observation points, flight geometries, metadata, and links to derived products. Initial ingestion targets newly processed historical airborne surveys; additional derived products will be incorporated as processing continues. Picking and tracing tools produce geolocated picks, travel-time or power measurements, ice-thickness estimates, and quality-control records. New collections require manifests and ingestion adapters rather than model execution adapters.

LIVIST explores Antarctic englacial-temperature products inferred from radar and constrained by borehole measurements. Its specialized frontend and documentation are built and served through CryoStack's deployment registry. LIVIST and Frozen Legacies are data-oriented rather than workflows organized around Local, Remote, or Cloud execution; data-serving interfaces and adapters can expose external repositories without relocating data to CryoStack.

Remote execution and the Connector/Relay

CryoLauncher and ICESEE bring existing institutional clusters or workstations into workflows through separate scheduler-aware submission, status, logging, termination, and retrieval implementations. **Direct** access uses SSH and file transfer from the application host when network policy and reachability allow. **Connector** serves resources reachable only from a workstation, VPN, or approved edge environment: a packaged Connector exchanges commands and files through an authenticated relay without inbound CryoStack connections to the institution. Institutional accounts, authentication, scheduler policies, and allocation enforcement remain authoritative.

Connector/Relay is connectivity infrastructure, also supporting Cloud access to private institutional services (Figure 2). Demonstrated CryoLauncher ISSM jobs on AWS Batch use it to reach Georgia Tech's MATLAB network-license service without exposing that service directly to the cloud. Users supply the license endpoint; CryoStack handles cloud-side license configuration and connectivity. Other Cloud workflows need no Connector unless private connectivity is required.

DRAFT

The Journal of Open Source Software

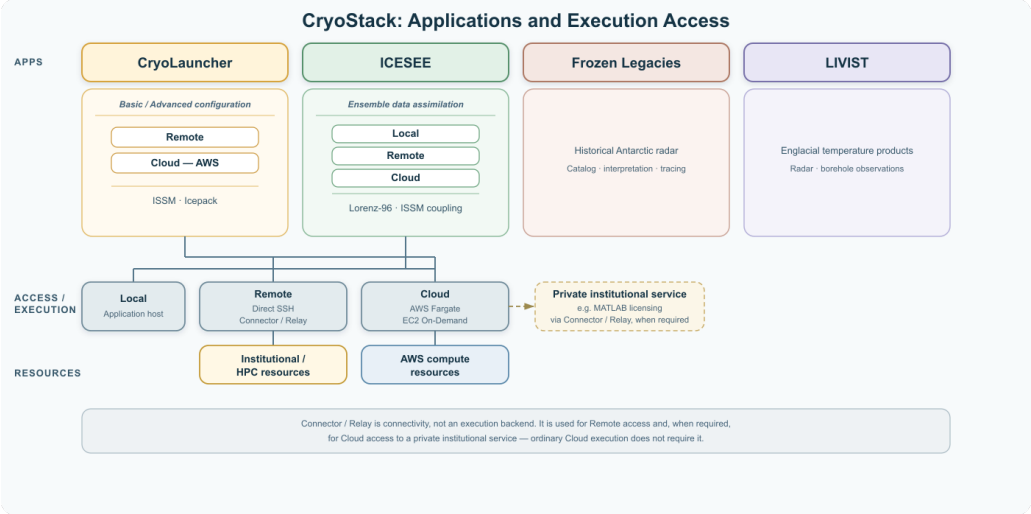


Figure 2: Application-specific execution and shared connectivity. Direct SSH or Connector/Relay reaches institutional resources; the dashed route supports Cloud access to private services. Observational applications retain specialized interfaces.

Cloud execution on AWS

Provider-independent driver contracts currently support AWS, with role-based personal-account onboarding. Fargate is the default AWS Batch compute mode; EC2 is optional. Supported workflows retain scientific configurations between Remote and Cloud execution, with different resource validation.

CryoLauncher Icepack workflows have completed Fargate and EC2 On-Demand execution on one CPU node. ISSM has completed both paths with MPI-parallel solvers, result packaging, retrieval, visualization, and Connector/Relay institutional MATLAB licensing. ICESEE's separate Cloud path has demonstrated single-process Lorenz-96 on Fargate. These qualify specific workflows, not EC2 Spot, custom networking, GPUs, or multi-node execution.

ICESEE-Spack resolves source builds against site compilers, MPI, and libraries using Spack (Gamblin et al., 2015). ICESEE-Containers provides preconfigured Docker and Apptainer environments; Apptainer follows the scientific-container approach described by Kurtzer et al. (2017).

Auto-config · Beta

Auto-config produces deterministic configuration proposals for CryoLauncher and ICESEE from English-language requests using the current configuration and registered model, example, parameter, resource, and capability metadata. Lexical matching and rule-based extraction follow constrained natural-language approaches (Caruso et al., 2024; Rutledge et al., 2022), combined with capability validation and provenance tracking. Proposals are presented for user review. Previews distinguish requested changes, suggestions, retained settings, and context from named workspace examples. Unspecified settings are retained where compatible; ambiguous or unsupported choices are not silently resolved.

Registered constraints support read-only diagnosis, explanations, and deterministic repair proposals. Refinement uses current controls, not conversation history. Applying a proposal updates the configuration only and rejects stale or altered proposals. Manual controls, validation, and execution remain authoritative; Auto-config cannot submit jobs, provision infrastructure, or configure licenses.

DRAFT

Results, packaging, and visualization

Standalone CryoLauncher ISSM and Icepack runs export transport-neutral, versioned result packages containing metadata, mesh geometry, field arrays, and available figures. Tutorials without plottable fields record export limitations without failing successful execution. Schema-specific readers satisfy a shared result-package protocol and support field and time-series visualization without MATLAB or Firedrake. Additional models can supply conforming exporters and readers. Icepack records conversion of Firedrake fields to a first-order nodal representation for display; validation of this conversion on live Firedrake output remains pending. ICESEE retains assimilation diagnostics; observational applications retain specialized results interfaces.

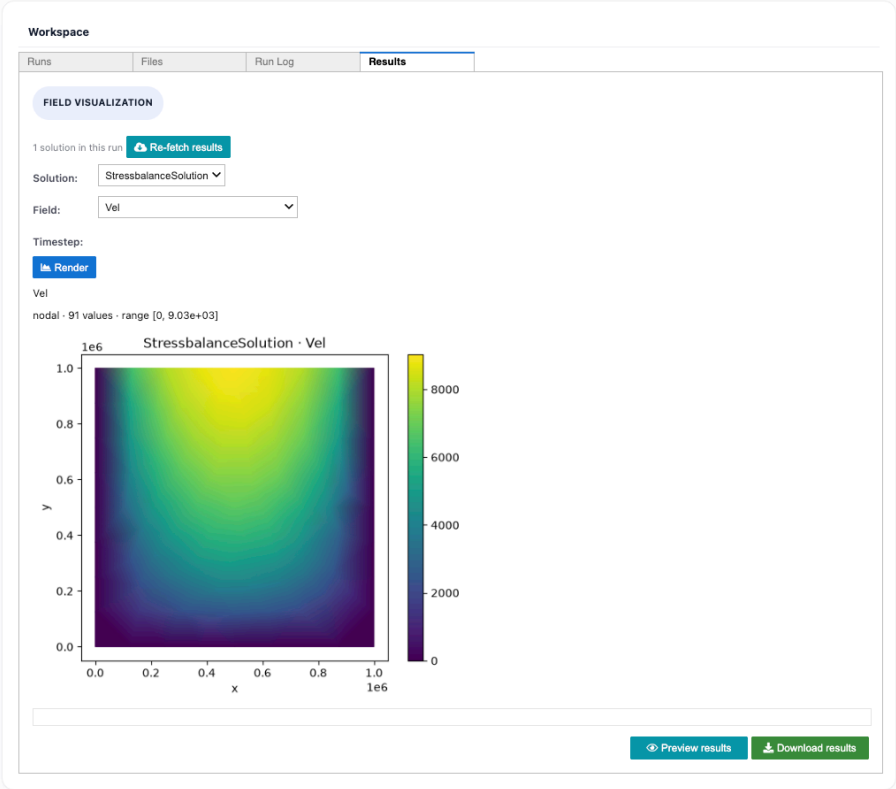


Figure 3: CryoLauncher displays ISSM stress-balance velocity magnitude (`StressbalanceSolution`, `Vel`), with solution and field selection, rendering, preview, and download controls.

Availability, verification, and limitations

Tests cover the gateway, authentication, adapters, Connector/Relay, cloud, and application-specific execution. Offline acceptance checks Auto-config safety, capability-registry consistency, and workspace isolation. These require no live infrastructure and neither establish scientific correctness nor replace live workflow qualification.

Limitations include process-local Connector sessions, incomplete resource-policy and audit enforcement, and qualification needed for shared AWS accounts and failure recovery. SQLite supports one-node deployment; scaling requires shared persistence and durable task and session storage. Connector binaries must track relay compatibility; reference images need project-registry publication. Additional workflows and Cloud configurations require qualification; result support and archival provenance remain incomplete. Not every Frozen Legacies desktop workflow is browser-native or provenance-captured. CryoStack uses the MIT License; release

1.0.0 is archived on Zenodo (Kyanjo, Robel, Tarzona, et al., 2026).

Research impact statement

Evidence is functional and architectural rather than adoption-based: CryoStack integrates the modeling, assimilation, and observational applications described above. Concrete examples include qualified remote-HPC and AWS workflows, deployed radar, and temperature interfaces. Research groups gain a common entry point for moving supported computational workflows among available resources and connecting observational applications to shared deployment and data infrastructure. Application registrations, model adapters and result readers, and dataset manifests and ingestion adapters support extension without a universal execution or result-handling model.

AI usage disclosure

Codex (OpenAI, GPT-5.6) and Claude (Anthropic, Sonnet 5) assisted with code documentation and selected user interface development. The authors reviewed, edited, and validated AI-assisted outputs and made the core design decisions.

Conflicts of interest

The authors declare no conflicts of interest.

Acknowledgements

This work was supported in part by the U.S. National Science Foundation through CAREER Award No. 2235920. We gratefully acknowledge the Astera Institute for funding the backend infrastructure for the LIVIST project. The authors also thank Renette Jones-Ivey of the University at Buffalo for her assistance with the initial Jupyter Book backend; Justin Quinn for helping launch the CryoStack-GHUB backends before their migration to a Georgia Tech virtual machine; Troy Hilley of Research Computing Services; and the Partnership for an Advanced Computing Environment (PACE) for providing computational resources. We further thank Research Scientist Fang (Cherry) Liu for her assistance in resolving challenges related to user authentication and remote access to PACE.

References

Caruso, S., Dodaro, C., Maratea, M., Mochi, M., & Riccio, F. (2024). CNL2ASP: Converting controlled natural language sentences into ASP. *Theory and Practice of Logic Programming*, *24*(2), 196–226. https://doi.org/10.1017/S1471068423000388

Gaffney, K. P., Prammer, M., Brasfield, L., Hipp, D. R., Kennedy, D., & Patel, J. M. (2022). SQLite: Past, present, and future. *Proceedings of the VLDB Endowment*, *15*(12), 3535–3547. https://doi.org/10.14778/3554821.3554842

Gamblin, T., LeGendre, M., Collette, M. R., Lee, G. L., Moody, A., Supinski, B. R. de, & Futral, S. (2015). The Spack package manager: Bringing order to HPC software chaos. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. https://doi.org/10.1145/2807591.2807623

Hudak, D., Johnson, D., Chalker, A., Nicklas, J., Franz, E., Dockendorf, T., & McMichael, B. L. (2018). Open OnDemand: A web-based client portal for HPC centers. *Journal of Open Source Software*, *3*(25), 622. https://doi.org/10.21105/joss.00622

DRAFT

Kurtzer, G. M., Sochat, V., & Bauer, M. W. (2017). Singularity: Scientific containers for mobility of compute. *PLOS ONE*, *12*(5), e0177459. https://doi.org/10.1371/journal.pone.0177459

Kyanjo, B., Mayo, T. L., & Robel, A. A. (2026). The ice sheet state and parameter estimator (ICESEE) library (v1.0.0): Ensemble kalman filtering for ice sheet models. *arXiv*. https://doi.org/10.48550/arXiv.2603.26947

Kyanjo, B., Robel, A. A., Tarzona, A., Dawson, E., & Chu, W. (2026). *CryoStack: A modular cyberinfrastructure stack for cryosphere data, models, data assimilation, and heterogeneous computing* (Version 1.0.0) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.22901225

Kyanjo, B., Robel, A., & Mayo, T. L. (2026). *ICESEE(v1.0.0): Ice sheet state and parameter estimator* (Version v1.0.0-v2). Zenodo. https://doi.org/10.5281/zenodo.21796612

Larour, E., Seroussi, H., Morlighem, M., & Rignot, E. (2012). Continental scale, high order, high spatial resolution ice sheet modeling using the Ice Sheet System Model (ISSM). *Journal of Geophysical Research: Earth Surface*, *117*(F1). https://doi.org/10.1029/2011JF002140

Project Jupyter Contributors. (n.d.). *JupyterHub documentation*. Retrieved October 20, 2026, from https://jupyterhub.readthedocs.io/en/stable/

Rutledge, L. W., Corbijn, J., Cuijpers, B., & Wondal, L. (2022). Rapid prototyping of business rule-based systems with controlled natural language and semantic web software. *Business Modeling and Software Design*, *453*, 3–20. https://doi.org/10.1007/978-3-031-11510-3_1

Schroeder, D. M., Dowdeswell, J. A., Siegert, M. J., Bingham, R. G., Chu, W., MacKie, E. J., Siegfried, M. R., Vega, K. I., Emmons, J. R., & Winstein, K. (2019). Multidecadal observations of the antarctic ice sheet from restored analog radar records. *Proceedings of the National Academy of Sciences*, *116*(38), 18867–18873. https://doi.org/10.1073/pnas.1821646116

Shapero, D. R., Badgeley, J. A., Hoffman, A. O., & Joughin, I. R. (2021). icepack: A new glacier flow modeling package in Python, version 1.0. *Geoscientific Model Development*, *14*(7), 4593–4616. https://doi.org/10.5194/gmd-14-4593-2021

DRAFT